

2.6.7 Newer Algorithms

Analogous to the newer methods for eigenvalues of symmetric tridiagonal matrices mentioned in §11.4.4, there are newer methods for SVD. There is a divide-and-conquer algorithm, implemented in LAPACK as `dgesdd`, which is typically faster by a factor of about 5 for large matrices than the algorithm we give (which is similar to the LAPACK routine `dgesvd`). Another routine based on the MRRR algorithm (see §11.4.4) promises to be even better, but it is not available in LAPACK as of 2006. It will appear as routine `dbdscr`.

CITED REFERENCES AND FURTHER READING:

- Numerical Recipes Software 2007, “SVD Implementation Code,” *Numerical Recipes Webnote No. 2*, at <http://www.nr.com/webnotes?2> [1]
- Forsythe, G.E., Malcolm, M.A., and Moler, C.B. 1977, *Computer Methods for Mathematical Computations* (Englewood Cliffs, NJ: Prentice-Hall), Chapter 9.[2]
- Golub, G.H., and Van Loan, C.F. 1996, *Matrix Computations*, 3rd ed. (Baltimore: Johns Hopkins University Press), §8.6 and Chapter 12 (SVD). QR decomposition is discussed in §5.2.6.[3]
- Lawson, C.L., and Hanson, R. 1974, *Solving Least Squares Problems* (Englewood Cliffs, NJ: Prentice-Hall); reprinted 1995 (Philadelphia: S.I.A.M.), Chapter 18.
- Wilkinson, J.H., and Reinsch, C. 1971, *Linear Algebra*, vol. II of *Handbook for Automatic Computation* (New York: Springer), Chapter I.10 by G.H. Golub and C. Reinsch.[4]
- Anderson, E., et al. 1999, LAPACK User's Guide, 3rd ed. (Philadelphia: S.I.A.M.). Online with software at 2007+, <http://www.netlib.org/lapack>.[5]
- Smith, B.T., et al. 1976, *Matrix Eigensystem Routines — EISPACK Guide*, 2nd ed., vol. 6 of *Lecture Notes in Computer Science* (New York: Springer).
- Stoer, J., and Bulirsch, R. 2002, *Introduction to Numerical Analysis*, 3rd ed. (New York: Springer), §6.7.[6]

2.7 Sparse Linear Systems

A system of linear equations is called *sparse* if only a relatively small number of its matrix elements a_{ij} are nonzero. It is wasteful to use general methods of linear algebra on such problems, because most of the $O(N^3)$ arithmetic operations devoted to solving the set of equations or inverting the matrix involve zero operands. Furthermore, you might wish to work problems so large as to tax your available memory space, and it is wasteful to reserve storage for unfruitful zero elements. Note that there are two distinct (and not always compatible) goals for any sparse matrix method: saving time and/or saving space.

We considered one archetypal sparse form in §2.4, the band-diagonal matrix. In the tridiagonal case, e.g., we saw that it was possible to save both time (order N instead of N^3) and space (order N instead of N^2). The method of solution was not different in principle from the general method of LU decomposition; it was just applied cleverly, and with due attention to the bookkeeping of zero elements. Many practical schemes for dealing with sparse problems have this same character. They are fundamentally decomposition schemes, or else elimination schemes akin to Gauss-Jordan, but carefully optimized so as to minimize the number of so-called

fill-ins, initially zero elements that must become nonzero during the solution process, and for which storage must be reserved.

Direct methods for solving sparse equations, then, depend crucially on the precise pattern of sparsity of the matrix. Patterns that occur frequently, or that are useful as way stations in the reduction of more general forms, already have special names and special methods of solution. We do not have space here for any detailed review of these. References listed at the end of this section will furnish you with an “in” to the specialized literature, and the following list of buzz words (and Figure 2.7.1) will at least let you hold your own at cocktail parties:

- tridiagonal
- band-diagonal (or banded) with bandwidth M
- band triangular
- block diagonal
- block tridiagonal
- block triangular
- cyclic banded
- singly (or doubly) bordered block diagonal
- singly (or doubly) bordered block triangular
- singly (or doubly) bordered band-diagonal
- singly (or doubly) bordered band triangular
- other (!)

You should also be aware of some of the special sparse forms that occur in the solution of partial differential equations in two or more dimensions. See Chapter 20.

If your particular pattern of sparsity is not a simple one, then you may wish to try an *analyze/factorize/operate* package, which automates the procedure of figuring out how fill-ins are to be minimized. The *analyze* stage is done once only for each pattern of sparsity. The *factorize* stage is done once for each particular matrix that fits the pattern. The *operate* stage is performed once for each right-hand side to be used with the particular matrix. Consult [2,3] for references on this. The NAG library [4] has an *analyze/factorize/operate* capability. A substantial collection of routines for sparse matrix calculation is also available from IMSL [5] as the *Yale Sparse Matrix Package* [6].

You should be aware that the special order of interchanges and eliminations, prescribed by a sparse matrix method so as to minimize fill-ins and arithmetic operations, generally acts to decrease the method’s numerical stability as compared to, e.g., regular *LU* decomposition with pivoting. Scaling your problem so as to make its nonzero matrix elements have comparable magnitudes (if you can do it) will sometimes ameliorate this problem.

In the remainder of this section, we present some concepts that are applicable to some general classes of sparse matrices, and which do not necessarily depend on details of the pattern of sparsity.

2.7.1 Sherman-Morrison Formula

Suppose that you have already obtained, by herculean effort, the inverse matrix $\mathbf{A}^{-1}$ of a square matrix $\mathbf{A}$. Now you want to make a “small” change in $\mathbf{A}$, for example change one element a_{ij} , or a few elements, or one row, or one column. Is there any

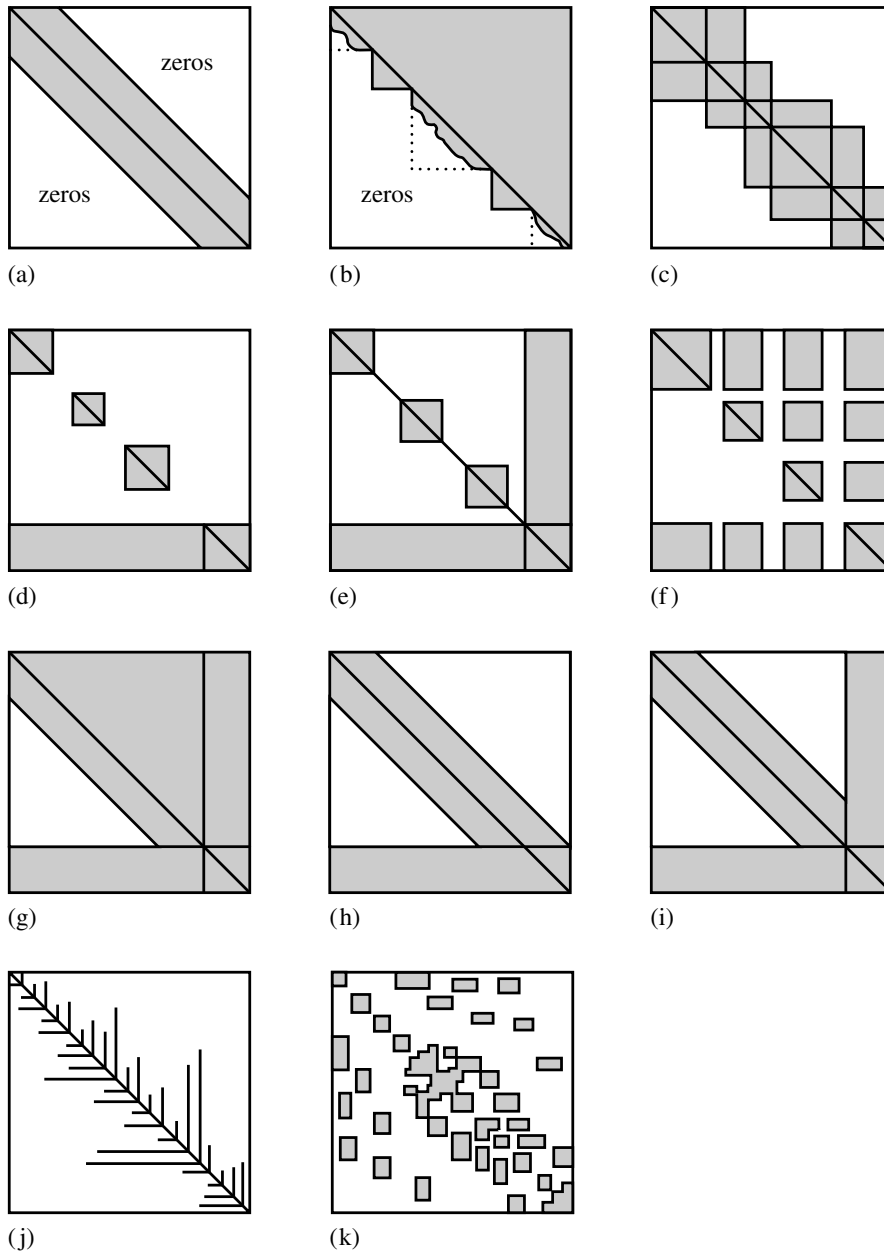


Figure 2.7.1. Some standard forms for sparse matrices. (a) Band-diagonal; (b) block triangular; (c) block tridiagonal; (d) singly bordered block diagonal; (e) doubly bordered block diagonal; (f) singly bordered block triangular; (g) bordered band-triangular; (h) and (i) singly and doubly bordered band-diagonal; (j) and (k) other! (after Tewarson) [1].

way of calculating the corresponding change in $\mathbf{A}^{-1}$ without repeating your difficult labors? Yes, if your change is of the form

$$\mathbf{A} \rightarrow (\mathbf{A} + \mathbf{u} \otimes \mathbf{v}) \quad (2.7.1)$$

for some vectors $\mathbf{u}$ and $\mathbf{v}$. If $\mathbf{u}$ is a unit vector $\mathbf{e}_i$, then (2.7.1) adds the components of $\mathbf{v}$ to the i th row. (Recall that $\mathbf{u} \otimes \mathbf{v}$ is a matrix whose i, j th element is the product of the i th component of $\mathbf{u}$ and the j th component of $\mathbf{v}$.) If $\mathbf{v}$ is a unit vector $\mathbf{e}_j$, then (2.7.1) adds the components of $\mathbf{u}$ to the j th column. If both $\mathbf{u}$ and $\mathbf{v}$ are proportional to unit vectors $\mathbf{e}_i$ and $\mathbf{e}_j$, respectively, then a term is added only to the element a_{ij} .

The *Sherman-Morrison* formula gives the inverse $(\mathbf{A} + \mathbf{u} \otimes \mathbf{v})^{-1}$ and is derived briefly as follows:

$$\begin{aligned}
 (\mathbf{A} + \mathbf{u} \otimes \mathbf{v})^{-1} &= (\mathbf{1} + \mathbf{A}^{-1} \cdot \mathbf{u} \otimes \mathbf{v})^{-1} \cdot \mathbf{A}^{-1} \\
 &= (\mathbf{1} - \mathbf{A}^{-1} \cdot \mathbf{u} \otimes \mathbf{v} + \mathbf{A}^{-1} \cdot \mathbf{u} \otimes \mathbf{v} \cdot \mathbf{A}^{-1} \cdot \mathbf{u} \otimes \mathbf{v} - \dots) \cdot \mathbf{A}^{-1} \\
 &= \mathbf{A}^{-1} - \mathbf{A}^{-1} \cdot \mathbf{u} \otimes \mathbf{v} \cdot \mathbf{A}^{-1} (1 - \lambda + \lambda^2 - \dots) \\
 &= \mathbf{A}^{-1} - \frac{(\mathbf{A}^{-1} \cdot \mathbf{u}) \otimes (\mathbf{v} \cdot \mathbf{A}^{-1})}{1 + \lambda}
 \end{aligned} \tag{2.7.2}$$

where

$$\lambda \equiv \mathbf{v} \cdot \mathbf{A}^{-1} \cdot \mathbf{u} \tag{2.7.3}$$

The second line of (2.7.2) is a formal power series expansion. In the third line, the associativity of outer and inner products is used to factor out the scalars λ .

The use of (2.7.2) is this: Given $\mathbf{A}^{-1}$ and the vectors $\mathbf{u}$ and $\mathbf{v}$, we need only perform two matrix multiplications and a vector dot product,

$$\mathbf{z} \equiv \mathbf{A}^{-1} \cdot \mathbf{u} \quad \mathbf{w} \equiv (\mathbf{A}^{-1})^T \cdot \mathbf{v} \quad \lambda = \mathbf{v} \cdot \mathbf{z} \tag{2.7.4}$$

to get the desired change in the inverse

$$\mathbf{A}^{-1} \rightarrow \mathbf{A}^{-1} - \frac{\mathbf{z} \otimes \mathbf{w}}{1 + \lambda} \tag{2.7.5}$$

The whole procedure requires only $3N^2$ multiplies and a like number of adds (an even smaller number if $\mathbf{u}$ or $\mathbf{v}$ is a unit vector).

The Sherman-Morrison formula can be directly applied to a class of sparse problems. If you already have a fast way of calculating the inverse of $\mathbf{A}$ (e.g., a tridiagonal matrix or some other standard sparse form), then (2.7.4) – (2.7.5) allow you to build up to your related but more complicated form, adding for example a row or column at a time. Notice that you can apply the Sherman-Morrison formula more than once successively, using at each stage the most recent update of $\mathbf{A}^{-1}$ (equation 2.7.5). Of course, if you have to modify *every* row, then you are back to an N^3 method. The constant in front of the N^3 is only a few times worse than the better direct methods, but you have deprived yourself of the stabilizing advantages of pivoting — so be careful.

For some other sparse problems, the Sherman-Morrison formula cannot be directly applied for the simple reason that storage of the whole inverse matrix $\mathbf{A}^{-1}$ is not feasible. If you want to add only a single correction of the form $\mathbf{u} \otimes \mathbf{v}$ and solve the linear system

$$(\mathbf{A} + \mathbf{u} \otimes \mathbf{v}) \cdot \mathbf{x} = \mathbf{b} \tag{2.7.6}$$

then you proceed as follows. Using the fast method that is presumed available for the matrix $\mathbf{A}$, solve the two auxiliary problems

$$\mathbf{A} \cdot \mathbf{y} = \mathbf{b} \quad \mathbf{A} \cdot \mathbf{z} = \mathbf{u} \tag{2.7.7}$$

for the vectors $\mathbf{y}$ and $\mathbf{z}$. In terms of these,

$$\mathbf{x} = \mathbf{y} - \left[\frac{\mathbf{v} \cdot \mathbf{y}}{1 + (\mathbf{v} \cdot \mathbf{z})} \right] \mathbf{z} \quad (2.7.8)$$

as we see by multiplying (2.7.2) on the right by $\mathbf{b}$.

2.7.2 Cyclic Tridiagonal Systems

So-called *cyclic tridiagonal systems* occur quite frequently and are a good example of how to use the Sherman-Morrison formula in the manner just described. The equations have the form

$$\begin{bmatrix} b_0 & c_0 & 0 & \cdots & & & \beta \\ a_1 & b_1 & c_1 & \cdots & & & \\ & & \cdots & & & & \\ & & \cdots & a_{N-2} & b_{N-2} & c_{N-2} & \\ \alpha & & \cdots & 0 & a_{N-1} & b_{N-1} & \end{bmatrix} \cdot \begin{bmatrix} x_0 \\ x_1 \\ \cdots \\ x_{N-2} \\ x_{N-1} \end{bmatrix} = \begin{bmatrix} r_0 \\ r_1 \\ \cdots \\ r_{N-2} \\ r_{N-1} \end{bmatrix} \quad (2.7.9)$$

This is a tridiagonal system, except for the matrix elements α and β in the corners. Forms like this are typically generated by finite differencing differential equations with periodic boundary conditions (§20.4).

We use the Sherman-Morrison formula, treating the system as tridiagonal plus a correction. In the notation of equation (2.7.6), define vectors $\mathbf{u}$ and $\mathbf{v}$ to be

$$\mathbf{u} = \begin{bmatrix} \gamma \\ 0 \\ \vdots \\ 0 \\ \alpha \end{bmatrix} \quad \mathbf{v} = \begin{bmatrix} 1 \\ 0 \\ \vdots \\ 0 \\ \beta/\gamma \end{bmatrix} \quad (2.7.10)$$

Here γ is arbitrary for the moment. Then the matrix $\mathbf{A}$ is the tridiagonal part of the matrix in (2.7.9), with two terms modified:

$$b'_0 = b_0 - \gamma, \quad b'_{N-1} = b_{N-1} - \alpha\beta/\gamma \quad (2.7.11)$$

We now solve equations (2.7.7) with the standard tridiagonal algorithm and then get the solution from equation (2.7.8).

The routine `cyclic` below implements this algorithm. We choose the arbitrary parameter $\gamma = -b_0$ to avoid loss of precision by subtraction in the first of equations (2.7.11). In the unlikely event that this causes loss of precision in the second of these equations, you can make a different choice.

```
void cyclic(VecDoub_I &a, VecDoub_I &b, VecDoub_I &c, const Doub alpha,      tridag.h
           const Doub beta, VecDoub_I &r, VecDoub_O &x)
Solves for a vector x[0..n-1] the "cyclic" set of linear equations given by equation (2.7.9). a,
b, c, and r are input vectors, all dimensioned as [0..n-1], while alpha and beta are the corner
entries in the matrix. The input is not modified.
{
    Int i,n=a.size();
    Doub fact,gamma;
    if (n <= 2) throw("n too small in cyclic");
```

```

VecDoub bb(n),u(n),z(n);
gamma = -b[0];
bb[0]=b[0]-gamma;
bb[n-1]=b[n-1]-alpha*beta/gamma;
for (i=1;i<n-1;i++) bb[i]=b[i];
tridag(a,bb,c,r,x);
u[0]=gamma;
u[n-1]=alpha;
for (i=1;i<n-1;i++) u[i]=0.0;
tridag(a,bb,c,u,z);
fact=(x[0]+beta*x[n-1]/gamma)/
      (1.0+z[0]+beta*z[n-1]/gamma);
for (i=0;i<n;i++) x[i] -= fact*z[i];
}

```

Avoid subtraction error in forming $bb[0]$.
Set up the diagonal of the modified tridiagonal system.

Solve $\mathbf{A} \cdot \mathbf{x} = \mathbf{r}$.
Set up the vector $\mathbf{u}$.

Solve $\mathbf{A} \cdot \mathbf{z} = \mathbf{u}$.
Form $\mathbf{v} \cdot \mathbf{x} / (1 + \mathbf{v} \cdot \mathbf{z})$.

Now get the solution vector $\mathbf{x}$.

2.7.3 Woodbury Formula

If you want to add more than a single correction term, then you cannot use (2.7.8) repeatedly, since without storing a new $\mathbf{A}^{-1}$ you will not be able to solve the auxiliary problems (2.7.7) efficiently after the first step. Instead, you need the *Woodbury formula*, which is the block-matrix version of the Sherman-Morrison formula,

$$\begin{aligned}
 &(\mathbf{A} + \mathbf{U} \cdot \mathbf{V}^T)^{-1} \\
 &= \mathbf{A}^{-1} - \left[\mathbf{A}^{-1} \cdot \mathbf{U} \cdot (\mathbf{1} + \mathbf{V}^T \cdot \mathbf{A}^{-1} \cdot \mathbf{U})^{-1} \cdot \mathbf{V}^T \cdot \mathbf{A}^{-1} \right]
 \end{aligned} \quad (2.7.12)$$

Here $\mathbf{A}$ is, as usual, an $N \times N$ matrix, while $\mathbf{U}$ and $\mathbf{V}$ are $N \times P$ matrices with $P < N$ and usually $P \ll N$. The inner piece of the correction term may become clearer if written as the tableau,

$$\left[\begin{array}{c} \mathbf{U} \end{array} \right] \cdot \left[\mathbf{1} + \mathbf{V}^T \cdot \mathbf{A}^{-1} \cdot \mathbf{U} \right]^{-1} \cdot \left[\begin{array}{c} \mathbf{V}^T \end{array} \right] \quad (2.7.13)$$

where you can see that the matrix whose inverse is needed is only $P \times P$ rather than $N \times N$.

The relation between the Woodbury formula and successive applications of the Sherman-Morrison formula is now clarified by noting that, if $\mathbf{U}$ is the matrix formed by columns out of the P vectors $\mathbf{u}_0, \dots, \mathbf{u}_{P-1}$, and $\mathbf{V}$ is the matrix formed by columns out of the P vectors $\mathbf{v}_0, \dots, \mathbf{v}_{P-1}$,

$$\mathbf{U} \equiv \left[\begin{array}{c} \mathbf{u}_0 \\ \vdots \\ \mathbf{u}_{P-1} \end{array} \right] \quad \mathbf{V} \equiv \left[\begin{array}{c} \mathbf{v}_0 \\ \vdots \\ \mathbf{v}_{P-1} \end{array} \right] \quad (2.7.14)$$

then two ways of expressing the same correction to $\mathbf{A}$ are

$$\left(\mathbf{A} + \sum_{k=0}^{P-1} \mathbf{u}_k \otimes \mathbf{v}_k \right) = (\mathbf{A} + \mathbf{U} \cdot \mathbf{V}^T) \quad (2.7.15)$$

(Note that the subscripts on $\mathbf{u}$ and $\mathbf{v}$ do *not* denote components, but rather distinguish the different column vectors.)

Equation (2.7.15) reveals that, if you have $\mathbf{A}^{-1}$ in storage, then you can either make the P corrections in one fell swoop by using (2.7.12) and inverting a $P \times P$ matrix, or else make them by applying (2.7.5) P successive times.

If you don't have storage for $\mathbf{A}^{-1}$, then you *must* use (2.7.12) in the following way: To solve the linear equation

$$\left(\mathbf{A} + \sum_{k=0}^{P-1} \mathbf{u}_k \otimes \mathbf{v}_k \right) \cdot \mathbf{x} = \mathbf{b} \quad (2.7.16)$$

first solve the P auxiliary problems

$$\begin{aligned} \mathbf{A} \cdot \mathbf{z}_0 &= \mathbf{u}_0 \\ \mathbf{A} \cdot \mathbf{z}_1 &= \mathbf{u}_1 \\ &\dots \\ \mathbf{A} \cdot \mathbf{z}_{P-1} &= \mathbf{u}_{P-1} \end{aligned} \quad (2.7.17)$$

and construct the matrix $\mathbf{Z}$ by columns from the $\mathbf{z}$'s obtained,

$$\mathbf{Z} \equiv \begin{bmatrix} \mathbf{z}_0 \\ \mathbf{z}_1 \\ \vdots \\ \mathbf{z}_{P-1} \end{bmatrix} \quad (2.7.18)$$

Next, do the $P \times P$ matrix inversion

$$\mathbf{H} \equiv (\mathbf{I} + \mathbf{V}^T \cdot \mathbf{Z})^{-1} \quad (2.7.19)$$

Finally, solve the one further auxiliary problem

$$\mathbf{A} \cdot \mathbf{y} = \mathbf{b} \quad (2.7.20)$$

In terms of these quantities, the solution is given by

$$\mathbf{x} = \mathbf{y} - \mathbf{Z} \cdot \left[\mathbf{H} \cdot (\mathbf{V}^T \cdot \mathbf{y}) \right] \quad (2.7.21)$$

2.7.4 Inversion by Partitioning

Once in a while, you will encounter a matrix (not even necessarily sparse) that can be inverted efficiently by partitioning. Suppose that the $N \times N$ matrix $\mathbf{A}$ is partitioned into

$$\mathbf{A} = \begin{bmatrix} \mathbf{P} & \mathbf{Q} \\ \mathbf{R} & \mathbf{S} \end{bmatrix} \quad (2.7.22)$$

where $\mathbf{P}$ and $\mathbf{S}$ are square matrices of size $p \times p$ and $s \times s$, respectively ($p + s = N$). The matrices $\mathbf{Q}$ and $\mathbf{R}$ are not necessarily square and have sizes $p \times s$ and $s \times p$, respectively.

If the inverse of $\mathbf{A}$ is partitioned in the same manner,

$$\mathbf{A}^{-1} = \begin{bmatrix} \tilde{\mathbf{P}} & \tilde{\mathbf{Q}} \\ \tilde{\mathbf{R}} & \tilde{\mathbf{S}} \end{bmatrix} \quad (2.7.23)$$

then $\tilde{\mathbf{P}}$, $\tilde{\mathbf{Q}}$, $\tilde{\mathbf{R}}$, $\tilde{\mathbf{S}}$, which have the same sizes as $\mathbf{P}$, $\mathbf{Q}$, $\mathbf{R}$, $\mathbf{S}$, respectively, can be found by either the formulas

$$\begin{aligned}\tilde{\mathbf{P}} &= (\mathbf{P} - \mathbf{Q} \cdot \mathbf{S}^{-1} \cdot \mathbf{R})^{-1} \\ \tilde{\mathbf{Q}} &= -(\mathbf{P} - \mathbf{Q} \cdot \mathbf{S}^{-1} \cdot \mathbf{R})^{-1} \cdot (\mathbf{Q} \cdot \mathbf{S}^{-1}) \\ \tilde{\mathbf{R}} &= -(\mathbf{S}^{-1} \cdot \mathbf{R}) \cdot (\mathbf{P} - \mathbf{Q} \cdot \mathbf{S}^{-1} \cdot \mathbf{R})^{-1} \\ \tilde{\mathbf{S}} &= \mathbf{S}^{-1} + (\mathbf{S}^{-1} \cdot \mathbf{R}) \cdot (\mathbf{P} - \mathbf{Q} \cdot \mathbf{S}^{-1} \cdot \mathbf{R})^{-1} \cdot (\mathbf{Q} \cdot \mathbf{S}^{-1})\end{aligned}\tag{2.7.24}$$

or else by the equivalent formulas

$$\begin{aligned}\tilde{\mathbf{P}} &= \mathbf{P}^{-1} + (\mathbf{P}^{-1} \cdot \mathbf{Q}) \cdot (\mathbf{S} - \mathbf{R} \cdot \mathbf{P}^{-1} \cdot \mathbf{Q})^{-1} \cdot (\mathbf{R} \cdot \mathbf{P}^{-1}) \\ \tilde{\mathbf{Q}} &= -(\mathbf{P}^{-1} \cdot \mathbf{Q}) \cdot (\mathbf{S} - \mathbf{R} \cdot \mathbf{P}^{-1} \cdot \mathbf{Q})^{-1} \\ \tilde{\mathbf{R}} &= -(\mathbf{S} - \mathbf{R} \cdot \mathbf{P}^{-1} \cdot \mathbf{Q})^{-1} \cdot (\mathbf{R} \cdot \mathbf{P}^{-1}) \\ \tilde{\mathbf{S}} &= (\mathbf{S} - \mathbf{R} \cdot \mathbf{P}^{-1} \cdot \mathbf{Q})^{-1}\end{aligned}\tag{2.7.25}$$

The parentheses in equations (2.7.24) and (2.7.25) highlight repeated factors that you may wish to compute only once. (Of course, by associativity, you can instead do the matrix multiplications in any order you like.) The choice between using equations (2.7.24) and (2.7.25) depends on whether you want $\tilde{\mathbf{P}}$ or $\tilde{\mathbf{S}}$ to have the simpler formula; or on whether the repeated expression $(\mathbf{S} - \mathbf{R} \cdot \mathbf{P}^{-1} \cdot \mathbf{Q})^{-1}$ is easier to calculate than the expression $(\mathbf{P} - \mathbf{Q} \cdot \mathbf{S}^{-1} \cdot \mathbf{R})^{-1}$; or on the relative sizes of $\mathbf{P}$ and $\mathbf{S}$; or on whether $\mathbf{P}^{-1}$ or $\mathbf{S}^{-1}$ is already known.

Another sometimes useful formula is for the determinant of the partitioned matrix,

$$\det \mathbf{A} = \det \mathbf{P} \det(\mathbf{S} - \mathbf{R} \cdot \mathbf{P}^{-1} \cdot \mathbf{Q}) = \det \mathbf{S} \det(\mathbf{P} - \mathbf{Q} \cdot \mathbf{S}^{-1} \cdot \mathbf{R})\tag{2.7.26}$$

2.7.5 Indexed Storage of Sparse Matrices

We have already seen (§2.4) that tri- or band-diagonal matrices can be stored in a compact format that allocates storage only to elements that can be nonzero, plus perhaps a few wasted locations to make the bookkeeping easier. What about more general sparse matrices? When a sparse matrix of dimension $M \times N$ contains only a few times M or N nonzero elements (a typical case), it is surely inefficient — and often physically impossible — to allocate storage for all MN elements. Even if one did allocate such storage, it would be inefficient or prohibitive in machine time to loop over all of it in search of nonzero elements.

Obviously some kind of indexed storage scheme is required, one that stores only nonzero matrix elements, along with sufficient auxiliary information to determine where an element logically belongs and how the various elements can be looped over in common matrix operations. Unfortunately, there is no one standard scheme in general use. Each scheme has its own pluses and minuses, depending on the application.

Before we look at sparse matrices, let's consider the simpler problem of a *sparse vector*. The obvious data structure is a list of the nonzero values and another list of the corresponding locations:

sparse.h

```
struct NRsparseCol
Sparse vector data structure.
{
    Int nrows;
    Int nvals;
```

Number of rows.
Maximum number of nonzeros.


```

VecInt row_ind;           Row indices of nonzeros.
VecDoub val;              Array of nonzero values.

NRsparseCol(Int m,Int nnvals) : nrows(m), nvals(nnvals),
row_ind(nnvals,0),val(nnvals,0.0) {}    Constructor. Initializes vector to zero.

NRsparseCol() : nrows(0),nvals(0),row_ind(),val() {}    Default constructor.

void resize(Int m, Int nnvals) {
    nrows = m;
    nvals = nnvals;
    row_ind.assign(nnvals,0);
    val.assign(nnvals,0.0);
}

};

```

While we think of this as defining a column vector, you can use exactly the same data structure for a row vector — just mentally interchange the meaning of row and column for the variables. For matrices, however, we have to decide ahead of time whether to use row-oriented or column-oriented storage.

One simple scheme is to use a vector of sparse columns:

```

NRvector<NRsparseCol *> a;
for (i=0;i<n;i++) {
    nvals=...
    a[i]=new NRSparseCol(m,nvals);
}

```

Each column is filled with statements like

```

count=0;
for (j=...) {
    a[i]->row_ind[count]=...
    a[i]->val[count]=...
    count++;
}

```

This data structure is good for an algorithm that primarily works with columns of the matrix, but it is not very efficient when one needs to loop over all elements of the matrix.

A good general storage scheme is the *compressed column storage* format. It is sometimes called the Harwell-Boeing format, after the two large organizations that first systematically provided a standard collection of sparse matrices for research purposes. In this scheme, three vectors are used: `val` for the nonzero values as they are traversed column by column, `row_ind` for the corresponding row indices of each value, and `col_ptr` for the locations in the other two arrays that start a column. In other words, if `val[k]=a[i][j]`, then `row_ind[k]=i`. The first nonzero in column `j` is at `col_ptr[j]`. The last is at `col_ptr[j+1]-1`. Note that `col_ptr[0]` is always 0, and by convention we define `col_ptr[n]` equal to the number of nonzeros. Note also that the dimension of the `col_ptr` array is $N + 1$, not N . The advantage of this scheme is that it requires storage of only about two times the number of nonzero matrix elements. (Other methods can require as much as three or five times.)

As an example, consider the matrix

$$\begin{bmatrix} 3.0 & 0.0 & 1.0 & 2.0 & 0.0 \\ 0.0 & 4.0 & 0.0 & 0.0 & 0.0 \\ 0.0 & 7.0 & 5.0 & 9.0 & 0.0 \\ 0.0 & 0.0 & 0.0 & 0.0 & 0.0 \\ 0.0 & 0.0 & 0.0 & 6.0 & 5.0 \end{bmatrix} \quad (2.7.27)$$

In compressed column storage mode, matrix (2.7.27) is represented by two arrays of length 9 and an array of length 6, as follows

index k	0	1	2	3	4	5	6	7	8
val[k]	3.0	4.0	7.0	1.0	5.0	2.0	9.0	6.0	5.0
row_ind[k]	0	1	2	0	2	0	2	4	4

(2.7.28)

index i	0	1	2	3	4	5
col_ptr[i]	0	1	3	5	8	9

Notice that, according to the storage rules, the value of N (namely 5) is the maximum valid index in `col_ptr`. The value of `col_ptr[5]` is 9, the length of the other two arrays. The elements 1.0 and 5.0 in column number 2, for example, are located in positions `col_ptr[2] ≤ k < col_ptr[3]`.

Here is a data structure to handle this storage scheme:

```
sparse.h struct NRsparseMat
Sparse matrix data structure for compressed column storage.
{
    Int nrows;           Number of rows.
    Int ncols;           Number of columns.
    Int nvals;           Maximum number of nonzeros.
    VecInt col_ptr;       Pointers to start of columns. Length is ncols+1.
    VecInt row_ind;       Row indices of nonzeros.
    VecDoub val;         Array of nonzero values.

    NRsparseMat();       Default constructor.
    NRsparseMat(Int m,Int n,Int nnvals); Constructor. Initializes vector to zero.
    VecDoub ax(const VecDoub &x) const; Multiply  $\mathbf{A}$  by a vector  $\mathbf{x}[0..ncols-1]$ .
    VecDoub atx(const VecDoub &x) const; Multiply  $\mathbf{A}^T$  by a vector  $\mathbf{x}[0..nrows-1]$ .
    NRsparseMat transpose() const; Form  $\mathbf{A}^T$ .
};
```

The code for the constructors is standard:

```
sparse.h NRsparseMat::NRsparseMat() : nrows(0),ncols(0),nvals(0),col_ptr(),
row_ind(),val() {}
NRsparseMat::NRsparseMat(Int m,Int n,Int nnvals) : nrows(m),ncols(n),
nvals(nnvals),col_ptr(n+1,0),row_ind(nnvals,0),val(nnvals,0.0) {}
```

The single most important use of a matrix in compressed column storage mode is to multiply a vector to its right. Don't implement this by traversing the rows of $\mathbf{A}$, which is extremely inefficient in this storage mode. Here's the right way to do it:

```
sparse.h VecDoub NRsparseMat::ax(const VecDoub &x) const {
    VecDoub y(nrows,0.0);
    for (Int j=0;j<ncols;j++) {
        for (Int i=col_ptr[j];i<col_ptr[j+1];i++)
            y[row_ind[i]] += val[i]*x[j];
    }
    return y;
}
```

Some inefficiency occurs because of the indirect addressing. While there are other storage modes that minimize this, they have their own drawbacks.

It is also simple to multiply the *transpose* of a matrix by a vector to its right, since we just traverse the columns directly. (Indirect addressing is still required.) Note that the transpose matrix is not actually constructed.

```
VecDoub NRsparseMat::atx(const VecDoub &x) const {
    VecDoub y(ncols);
    for (Int i=0;i<ncols;i++) {
        y[i]=0.0;
        for (Int j=col_ptr[i];j<col_ptr[i+1];j++)
            y[i] += val[j]*x[row_ind[j]];
    }
    return y;
}
```

sparse.h

Because the choice of compressed column storage treats rows and columns quite differently, it is rather an involved operation to construct the transpose of a matrix, given the matrix itself in compressed column storage mode. When the operation cannot be avoided, it is

```
NRsparseMat NRsparseMat::transpose() const {
    Int i,j,k,index,m=nrows,n=ncols;
    NRsparseMat at(n,m,nvals);           Initialized to zero.
    First find the column lengths for  $\mathbf{A}^T$ , i.e. the row lengths of  $\mathbf{A}$ .
    VecInt count(m,0);                   Temporary counters for each row of  $\mathbf{A}$ .
    for (i=0;i<n;i++)
        for (j=col_ptr[i];j<col_ptr[i+1];j++) {
            k=row_ind[j];
            count[k]++;
        }
    for (j=0;j<m;j++)                    Now set at.col_ptr. 0th entry stays 0.
        at.col_ptr[j+1]=at.col_ptr[j]+count[j];
    for(j=0;j<m;j++)                      Reset counters to zero.
        count[j]=0;
    for (i=0;i<n;i++)                    Main loop.
        for (j=col_ptr[i];j<col_ptr[i+1];j++) {
            k=row_ind[j];
            index=at.col_ptr[k]+count[k]; Element's position in column of  $\mathbf{A}^T$ .
            at.row_ind[index]=i;
            at.val[index]=val[j];
            count[k]++;                    Increment counter for next element in that
        }                                column.
    return at;
}
```

sparse.h

The only sparse matrix-matrix multiply routine we give is to form the product $\mathbf{A}\mathbf{D}\mathbf{A}^T$, where $\mathbf{D}$ is a diagonal matrix. This particular product is used to form the so-called normal equations in the interior-point method for linear programming (§10.11). We encapsulate the algorithm in its own structure, ADAT:

```
struct ADAT {
    const NRsparseMat &a,&at;              Store references to  $\mathbf{A}$  and  $\mathbf{A}^T$ .
    NRsparseMat *adat;                    This will hold  $\mathbf{A}\mathbf{D}\mathbf{A}^T$ .

    ADAT(const NRsparseMat &A,const NRsparseMat &AT);
    Allocates compressed column storage for  $\mathbf{A}\mathbf{A}^T$ , where  $\mathbf{A}$  and  $\mathbf{A}^T$  are input in compressed
    column format, and fills in values of col_ptr and row_ind. Each column must be in sorted
    order in input matrices. Matrix is output with each column sorted.
    void updateD(const VecDoub &D);
    Computes  $\mathbf{A}\mathbf{D}\mathbf{A}^T$ , where  $\mathbf{D}$  is a diagonal matrix. This function can be called repeatedly
    to update  $\mathbf{A}\mathbf{D}\mathbf{A}^T$  for fixed  $\mathbf{A}$ .
    NRsparseMat &ref();
    Returns reference to adat, which holds  $\mathbf{A}\mathbf{D}\mathbf{A}^T$ .
    ~ADAT();
};
```

sparse.h

The algorithm proceeds in two steps. First, the nonzero pattern of $\mathbf{A}\mathbf{A}^T$ is found by a call to the constructor. Since $\mathbf{D}$ is diagonal, $\mathbf{A}\mathbf{A}^T$ and $\mathbf{A}\mathbf{D}\mathbf{A}^T$ have the same nonzero structure. Algorithms using ADAT will typically have both $\mathbf{A}$ and $\mathbf{A}^T$ available, so we pass them both to the constructor rather than recompute $\mathbf{A}^T$ from $\mathbf{A}$. The constructor allocates storage and assigns values to `col_ptr` and `row_ind`. The structure of ADAT is returned with columns in sorted order because routines like the AMD ordering algorithm used in §10.11 require it.

```

sparse.h  ADAT::ADAT(const NRsparseMat &A,const NRsparseMat &AT) : a(A), at(AT) {
    Int h,i,j,k,l,nvals,m=AT.ncols;
    VecInt done(m);
    for (i=0;i<m;i++)                                Initialize to not done.
        done[i]=-1;
    nvals=0;                                           First pass determines number of nonzeros.
    for (j=0;j<m;j++) {                               Outer loop over columns of  $\mathbf{A}^T$  in  $\mathbf{A}\mathbf{A}^T$ .
        for (i=AT.col_ptr[j];i<AT.col_ptr[j+1];i++) {
            k=AT.row_ind[i];                           $\mathbf{A}_{kj}^T \neq 0$ . Find column  $k$  in first matrix,  $\mathbf{A}$ .
            for (l=A.col_ptr[k];l<A.col_ptr[k+1];l++) {
                h=A.row_ind[l];                         $\mathbf{A}_{hl} \neq 0$ .
                if (done[h] != j) {                    Test if contribution already included.
                    done[h]=j;
                    nvals++;
                }
            }
        }
    }
    adat = new NRsparseMat(m,m,nvals);                Allocate storage for ADAT.
    for (i=0;i<m;i++)                                Re-initialize.
        done[i]=-1;
    nvals=0;
    Second pass: Determine columns of adat. Code is identical to first pass except adat->col_ptr
    and adat->row_ind get assigned at appropriate places.
    for (j=0;j<m;j++) {
        adat->col_ptr[j]=nvals;
        for (i=AT.col_ptr[j];i<AT.col_ptr[j+1];i++) {
            k=AT.row_ind[i];
            for (l=A.col_ptr[k];l<A.col_ptr[k+1];l++) {
                h=A.row_ind[l];
                if (done[h] != j) {
                    done[h]=j;
                    adat->row_ind[nvals]=h;
                    nvals++;
                }
            }
        }
    }
    adat->col_ptr[m]=nvals;                            Set last value.
    for (j=0;j<m;j++) {                                Sort columns
        i=adat->col_ptr[j];
        Int size=adat->col_ptr[j+1]-i;
        if (size > 1) {
            VecInt col(size,&adat->row_ind[i]);
            sort(col);
            for (k=0;k<size;k++)
                adat->row_ind[i+k]=col[k];
        }
    }
}

```

The next routine, `updatedD`, actually fills in the values in the `val` array. It can be called repeatedly to update $\mathbf{A}\mathbf{D}\mathbf{A}^T$ for fixed $\mathbf{A}$.

```

void ADAT::updateD(const VecDoub &D) {
    Int h,i,j,k,l,m=a.nrows,n=a.ncols;
    VecDoub temp(n),temp2(m,0.0);
    for (i=0;i<m;i++) {
        for (j=at.col_ptr[i];j< at.col_ptr[i+1];j++) {
            k=at.row_ind[j];
            temp[k]=at.val[j]*D[k];
        }
        for (j=at.col_ptr[i];j<at.col_ptr[i+1];j++) {
            k=at.row_ind[j];
            for (l=a.col_ptr[k];l<a.col_ptr[k+1];l++) {
                h=a.row_ind[l];
                temp2[h] += temp[k]*a.val[l];
            }
        }
        for (j=adat->col_ptr[i];j<adat->col_ptr[i+1];j++) {
            k=adat->row_ind[j];
            adat->val[j]=temp2[k];
            temp2[k]=0.0;
        }
    }
}

```

sparse.h

Outer loop over columns of $\mathbf{A}^T$.
 Scale elements of each column with $\mathbf{D}$ and store in temp.
 Go down column again.
 Go down column k in $\mathbf{A}$.
 All terms from temp[k] used here.
 Store temp2 in column of answer.
 Restore temp2.

The final two functions are simple. The ref routine returns a reference to the matrix $\mathbf{ADAT}^T$ stored in the structure for other routines that may need to work with it. And the destructor releases the storage.

```

NRsparseMat & ADAT::ref() {
    return *adat;
}

ADAT::~~ADAT() {
    delete adat;
}

```

sparse.h

By the way, if you invoke ADAT with *different* matrices $\mathbf{A}$ and $\mathbf{B}^T$, everything will work fine as long as $\mathbf{A}$ and $\mathbf{B}$ have the same nonzero pattern.

In *Numerical Recipes* second edition, we gave a related sparse matrix storage mode in which the diagonal of the matrix is stored first, followed by the off-diagonal elements. We now feel that the added complexity of that scheme is not worthwhile for any of the uses in this book. For a discussion of this and other storage schemes, see [7,8]. To see how to work with the diagonal in the compressed column mode, look at the code for `asolve` at the end of this section.

2.7.6 Conjugate Gradient Method for a Sparse System

So-called *conjugate gradient methods* provide a quite general means for solving the $N \times N$ linear system

$$\mathbf{A} \cdot \mathbf{x} = \mathbf{b} \quad (2.7.29)$$

The attractiveness of these methods for large sparse systems is that they reference $\mathbf{A}$ only through its multiplication of a vector, or the multiplication of its transpose and a vector. As we have seen, these operations can be very efficient for a properly stored sparse matrix. You, the “owner” of the matrix $\mathbf{A}$, can be asked to provide functions that perform these sparse matrix multiplications as efficiently as possible. We, the “grand strategists,” supply an abstract base class, `Linbcg` below, that contains the method for solving the set of linear equations, (2.7.29), using your functions.

The simplest, “ordinary” conjugate gradient algorithm [9-11] solves (2.7.29) only in the case that $\mathbf{A}$ is symmetric and positive-definite. It is based on the idea of minimizing the function

$$f(\mathbf{x}) = \frac{1}{2} \mathbf{x} \cdot \mathbf{A} \cdot \mathbf{x} - \mathbf{b} \cdot \mathbf{x} \quad (2.7.30)$$

This function is minimized when its gradient

$$\nabla f = \mathbf{A} \cdot \mathbf{x} - \mathbf{b} \quad (2.7.31)$$

is zero, which is equivalent to (2.7.29). The minimization is carried out by generating a succession of search directions $\mathbf{p}_k$ and improved minimizers $\mathbf{x}_k$. At each stage a quantity α_k is found that minimizes $f(\mathbf{x}_k + \alpha_k \mathbf{p}_k)$, and $\mathbf{x}_{k+1}$ is set equal to the new point $\mathbf{x}_k + \alpha_k \mathbf{p}_k$. The $\mathbf{p}_k$ and $\mathbf{x}_k$ are built up in such a way that $\mathbf{x}_{k+1}$ is also the minimizer of f over the whole vector space of directions already taken, $\{\mathbf{p}_0, \mathbf{p}_1, \dots, \mathbf{p}_{k-1}\}$. After N iterations you arrive at the minimizer over the entire vector space, i.e., the solution to (2.7.29).

Later, in §10.8, we will generalize this “ordinary” conjugate gradient algorithm to the minimization of arbitrary nonlinear functions. Here, where our interest is in solving linear, but not necessarily positive-definite or symmetric, equations, a different generalization is important, the *biconjugate gradient method*. This method does not, in general, have a simple connection with function minimization. It constructs four sequences of vectors, $\mathbf{r}_k, \bar{\mathbf{r}}_k, \mathbf{p}_k, \bar{\mathbf{p}}_k, k = 0, 1, \dots$. You supply the initial vectors $\mathbf{r}_0$ and $\bar{\mathbf{r}}_0$, and set $\mathbf{p}_0 = \mathbf{r}_0, \bar{\mathbf{p}}_0 = \bar{\mathbf{r}}_0$. Then you carry out the following recurrence:

$$\begin{aligned} \alpha_k &= \frac{\bar{\mathbf{r}}_k \cdot \mathbf{r}_k}{\bar{\mathbf{p}}_k \cdot \mathbf{A} \cdot \mathbf{p}_k} \\ \mathbf{r}_{k+1} &= \mathbf{r}_k - \alpha_k \mathbf{A} \cdot \mathbf{p}_k \\ \bar{\mathbf{r}}_{k+1} &= \bar{\mathbf{r}}_k - \alpha_k \mathbf{A}^T \cdot \bar{\mathbf{p}}_k \\ \beta_k &= \frac{\bar{\mathbf{r}}_{k+1} \cdot \mathbf{r}_{k+1}}{\bar{\mathbf{r}}_k \cdot \mathbf{r}_k} \\ \mathbf{p}_{k+1} &= \mathbf{r}_{k+1} + \beta_k \mathbf{p}_k \\ \bar{\mathbf{p}}_{k+1} &= \bar{\mathbf{r}}_{k+1} + \beta_k \bar{\mathbf{p}}_k \end{aligned} \quad (2.7.32)$$

This sequence of vectors satisfies the *biorthogonality* condition

$$\bar{\mathbf{r}}_i \cdot \mathbf{r}_j = \mathbf{r}_i \cdot \bar{\mathbf{r}}_j = 0, \quad j < i \quad (2.7.33)$$

and the *biconjugacy* condition

$$\bar{\mathbf{p}}_i \cdot \mathbf{A} \cdot \mathbf{p}_j = \mathbf{p}_i \cdot \mathbf{A}^T \cdot \bar{\mathbf{p}}_j = 0, \quad j < i \quad (2.7.34)$$

There is also a mutual orthogonality,

$$\bar{\mathbf{r}}_i \cdot \mathbf{p}_j = \mathbf{r}_i \cdot \bar{\mathbf{p}}_j = 0, \quad j < i \quad (2.7.35)$$

The proof of these properties proceeds by straightforward induction [12]. As long as the recurrence does not break down earlier because one of the denominators is zero, it must terminate after $m \leq N$ steps with $\mathbf{r}_m = \bar{\mathbf{r}}_m = 0$. This is basically because after at most N steps you run out of new orthogonal directions to the vectors you’ve already constructed.

To use the algorithm to solve the system (2.7.29), make an initial guess $\mathbf{x}_0$ for the solution. Choose $\mathbf{r}_0$ to be the *residual*

$$\mathbf{r}_0 = \mathbf{b} - \mathbf{A} \cdot \mathbf{x}_0 \quad (2.7.36)$$

and choose $\bar{\mathbf{r}}_0 = \mathbf{r}_0$. Then form the sequence of improved estimates

$$\mathbf{x}_{k+1} = \mathbf{x}_k + \alpha_k \mathbf{p}_k \quad (2.7.37)$$

while carrying out the recurrence (2.7.32). Equation (2.7.37) guarantees that $\mathbf{r}_{k+1}$ from the recurrence is in fact the residual $\mathbf{b} - \mathbf{A} \cdot \mathbf{x}_{k+1}$ corresponding to $\mathbf{x}_{k+1}$. Since $\mathbf{r}_m = 0$, $\mathbf{x}_m$ is the solution to equation (2.7.29).

While there is no guarantee that this whole procedure will not break down or become unstable for general $\mathbf{A}$, in practice this is rare. More importantly, the exact termination in at most N iterations occurs only with exact arithmetic. Roundoff error means that you should

regard the process as a genuinely iterative procedure, to be halted when some appropriate error criterion is met.

The ordinary conjugate gradient algorithm is the special case of the biconjugate gradient algorithm when $\mathbf{A}$ is symmetric, and we choose $\bar{\mathbf{r}}_0 = \mathbf{r}_0$. Then $\bar{\mathbf{r}}_k = \mathbf{r}_k$ and $\bar{\mathbf{p}}_k = \mathbf{p}_k$ for all k ; you can omit computing them and halve the work of the algorithm. This conjugate gradient version has the interpretation of minimizing equation (2.7.30). If $\mathbf{A}$ is positive-definite as well as symmetric, the algorithm cannot break down (in theory!). The `solve` routine `Linbcg` below indeed reduces to the ordinary conjugate gradient method if you input a symmetric $\mathbf{A}$, but it does all the redundant computations.

Another variant of the general algorithm corresponds to a symmetric but non-positive-definite $\mathbf{A}$, with the choice $\bar{\mathbf{r}}_0 = \mathbf{A} \cdot \mathbf{r}_0$ instead of $\bar{\mathbf{r}}_0 = \mathbf{r}_0$. In this case $\bar{\mathbf{r}}_k = \mathbf{A} \cdot \mathbf{r}_k$ and $\bar{\mathbf{p}}_k = \mathbf{A} \cdot \mathbf{p}_k$ for all k . This algorithm is thus equivalent to the ordinary conjugate gradient algorithm, but with all dot products $\mathbf{a} \cdot \mathbf{b}$ replaced by $\mathbf{a} \cdot \mathbf{A} \cdot \mathbf{b}$. It is called the *minimum residual* algorithm, because it corresponds to successive minimizations of the function

$$\Phi(\mathbf{x}) = \frac{1}{2} \mathbf{r} \cdot \mathbf{r} = \frac{1}{2} |\mathbf{A} \cdot \mathbf{x} - \mathbf{b}|^2 \quad (2.7.38)$$

where the successive iterates $\mathbf{x}_k$ minimize Φ over the same set of search directions $\mathbf{p}_k$ generated in the conjugate gradient method. This algorithm has been generalized in various ways for unsymmetric matrices. The *generalized minimum residual* method (GMRES; see [13,14]) is probably the most robust of these methods.

Note that equation (2.7.38) gives

$$\nabla \Phi(\mathbf{x}) = \mathbf{A}^T \cdot (\mathbf{A} \cdot \mathbf{x} - \mathbf{b}) \quad (2.7.39)$$

For any nonsingular matrix $\mathbf{A}$, $\mathbf{A}^T \cdot \mathbf{A}$ is symmetric and positive-definite. You might therefore be tempted to solve equation (2.7.29) by applying the ordinary conjugate gradient algorithm to the problem

$$(\mathbf{A}^T \cdot \mathbf{A}) \cdot \mathbf{x} = \mathbf{A}^T \cdot \mathbf{b} \quad (2.7.40)$$

Don't! The condition number of the matrix $\mathbf{A}^T \cdot \mathbf{A}$ is the square of the condition number of $\mathbf{A}$ (see §2.6 for definition of condition number). A large condition number both increases the number of iterations required and limits the accuracy to which a solution can be obtained. It is almost always better to apply the biconjugate gradient method to the original matrix $\mathbf{A}$.

So far we have said nothing about the *rate* of convergence of these methods. The ordinary conjugate gradient method works well for matrices that are well-conditioned, i.e., “close” to the identity matrix. This suggests applying these methods to the *preconditioned* form of equation (2.7.29),

$$(\tilde{\mathbf{A}}^{-1} \cdot \mathbf{A}) \cdot \mathbf{x} = \tilde{\mathbf{A}}^{-1} \cdot \mathbf{b} \quad (2.7.41)$$

The idea is that you might already be able to solve your linear system easily for some $\tilde{\mathbf{A}}$ close to $\mathbf{A}$, in which case $\tilde{\mathbf{A}}^{-1} \cdot \mathbf{A} \approx \mathbf{1}$, allowing the algorithm to converge in fewer steps. The matrix $\tilde{\mathbf{A}}$ is called a *preconditioner* [9], and the overall scheme given here is known as the *preconditioned biconjugate gradient method* or *PBCG*. In the code below, the user-supplied routine `atimes` does sparse matrix multiplication by $\mathbf{A}$, while the user-supplied routine `asolve` effects matrix multiplication by the inverse of the preconditioner $\tilde{\mathbf{A}}^{-1}$.

For efficient implementation, the PBCG algorithm introduces an additional set of vectors $\mathbf{z}_k$ and $\bar{\mathbf{z}}_k$ defined by

$$\tilde{\mathbf{A}} \cdot \mathbf{z}_k = \mathbf{r}_k \quad \text{and} \quad \tilde{\mathbf{A}}^T \cdot \bar{\mathbf{z}}_k = \bar{\mathbf{r}}_k \quad (2.7.42)$$

and modifies the definitions of α_k , β_k , $\mathbf{p}_k$, and $\bar{\mathbf{p}}_k$ in equation (2.7.32):

$$\begin{aligned} \alpha_k &= \frac{\bar{\mathbf{r}}_k \cdot \mathbf{z}_k}{\bar{\mathbf{p}}_k \cdot \mathbf{A} \cdot \mathbf{p}_k} \\ \beta_k &= \frac{\bar{\mathbf{r}}_{k+1} \cdot \mathbf{z}_{k+1}}{\bar{\mathbf{r}}_k \cdot \mathbf{z}_k} \\ \mathbf{p}_{k+1} &= \mathbf{z}_{k+1} + \beta_k \mathbf{p}_k \\ \bar{\mathbf{p}}_{k+1} &= \bar{\mathbf{z}}_{k+1} + \beta_k \bar{\mathbf{p}}_k \end{aligned} \quad (2.7.43)$$

To use `Linbcg`, below, you will need to supply routines that solve the auxiliary linear systems (2.7.42). If you have no idea what to use for the preconditioner $\tilde{\mathbf{A}}$, then use the diagonal part of $\mathbf{A}$, or even the identity matrix, in which case the burden of convergence will be entirely on the biconjugate gradient method itself.

`Linbcg`'s routine `solve`, below, is based on a program originally written by Anne Greenbaum. (See [11] for a different, less sophisticated, implementation.) There are a few wrinkles you should know about.

What constitutes “good” convergence is rather application-dependent. The routine `solve` therefore provides for four possibilities, selected by setting the flag `itol` on input. If `itol=1`, iteration stops when the quantity $|\mathbf{A} \cdot \mathbf{x} - \mathbf{b}|/|\mathbf{b}|$ is less than the input quantity `tol`. If `itol=2`, the required criterion is

$$|\tilde{\mathbf{A}}^{-1} \cdot (\mathbf{A} \cdot \mathbf{x} - \mathbf{b})|/|\tilde{\mathbf{A}}^{-1} \cdot \mathbf{b}| < \text{tol} \quad (2.7.44)$$

If `itol=3`, the routine uses its own estimate of the error in $\mathbf{x}$ and requires its magnitude, divided by the magnitude of $\mathbf{x}$, to be less than `tol`. The setting `itol=4` is the same as `itol=3`, except that the largest (in absolute value) component of the error and largest component of $\mathbf{x}$ are used instead of the vector magnitude (that is, the L_∞ norm instead of the L_2 norm). You may need to experiment to find which of these convergence criteria is best for your problem.

On output, `err` is the tolerance actually achieved. If the returned count `iter` does not indicate that the maximum number of allowed iterations `itmax` was exceeded, then `err` should be less than `tol`. If you want to do further iterations, leave all returned quantities as they are and call the routine again. The routine loses its memory of the spanned conjugate gradient subspace between calls, however, so you should not force it to return more often than about every N iterations.

`linbcg.h`

```
struct Linbcg {
    Abstract base class for solving sparse linear equations by the preconditioned biconjugate gradient
    method. To use, declare a derived class in which the methods atimes and asolve are defined for
    your problem, along with any data that they need. Then call the solve method.
```

```
    virtual void asolve(VecDoub_I &b, VecDoub_O &x, const Int itrns) = 0;
    virtual void atimes(VecDoub_I &x, VecDoub_O &r, const Int itrns) = 0;
    void solve(VecDoub_I &b, VecDoub_IO &x, const Int itol, const Doub tol,
              const Int itmax, Int &iter, Doub &err);
    Doub snrm(VecDoub_I &sx, const Int itol);           Utility used by solve.
```

```
};
```

```
void Linbcg::solve(VecDoub_I &b, VecDoub_IO &x, const Int itol, const Doub tol,
                  const Int itmax, Int &iter, Doub &err)
```

Solves $\mathbf{A} \cdot \mathbf{x} = \mathbf{b}$ for $\mathbf{x}[0..n-1]$, given $\mathbf{b}[0..n-1]$, by the iterative biconjugate gradient method. On input $\mathbf{x}[0..n-1]$ should be set to an initial guess of the solution (or all zeros); `itol` is 1,2,3, or 4, specifying which convergence test is applied (see text); `itmax` is the maximum number of allowed iterations; and `tol` is the desired convergence tolerance. On output, $\mathbf{x}[0..n-1]$ is reset to the improved solution, `iter` is the number of iterations actually taken, and `err` is the estimated error. The matrix $\mathbf{A}$ is referenced only through the user-supplied routines `atimes`, which computes the product of either $\mathbf{A}$ or its transpose on a vector, and `asolve`, which solves $\tilde{\mathbf{A}} \cdot \mathbf{x} = \mathbf{b}$ or $\tilde{\mathbf{A}}^T \cdot \mathbf{x} = \mathbf{b}$ for some preconditioner matrix $\tilde{\mathbf{A}}$ (possibly the trivial diagonal part of $\mathbf{A}$). This routine can be called repeatedly, with `itmax` $\lesssim n$, to monitor how `err` decreases; or it can be called once with a sufficiently large value of `itmax` so that convergence to `tol` is achieved.

```
{
    Doub ak,akden,bk,bkden=1.0,bknum,bnrm,dxnrm,xnrm,zminrm,znrm;
    const Doub EPS=1.0e-14;
    Int j,n=b.size();
    VecDoub p(n),pp(n),r(n),rr(n),z(n),zz(n);
    iter=0;
    atimes(x,r,0);           Calculate initial residual.
    for (j=0;j<n;j++) {      Input to atimes is x[0..n-1], output is r[0..n-1];
        r[j]=b[j]-r[j];      the final 0 indicates that the matrix (not its
        rr[j]=r[j];          transpose) is to be used.
    }
}
```



```

//atimes(r,rr,0);
if (itol == 1) {
    bnorm=snrm(b,itol);
    asolve(r,z,0);
}
else if (itol == 2) {
    asolve(b,z,0);
    bnorm=snrm(z,itol);
    asolve(r,z,0);
}
else if (itol == 3 || itol == 4) {
    asolve(b,z,0);
    bnorm=snrm(z,itol);
    asolve(r,z,0);
    znrm=snrm(z,itol);
} else throw("illegal itol in linbcg");
while (iter < itmax) {
    ++iter;
    asolve(rr,zz,1);
    for (bknum=0.0,j=0;j<n;j++) bknum += z[j]*rr[j];
    Calculate coefficient bk and direction vectors p and pp.
    if (iter == 1) {
        for (j=0;j<n;j++) {
            p[j]=z[j];
            pp[j]=zz[j];
        }
    } else {
        bk=bknum/bkden;
        for (j=0;j<n;j++) {
            p[j]=bk*p[j]+z[j];
            pp[j]=bk*pp[j]+zz[j];
        }
    }
    bkden=bknum;
    atimes(p,z,0);
    for (akden=0.0,j=0;j<n;j++) akden += z[j]*pp[j];
    ak=bknum/akden;
    atimes(pp,zz,1);
    for (j=0;j<n;j++) {
        x[j] += ak*p[j];
        r[j] -= ak*z[j];
        rr[j] -= ak*zz[j];
    }
    asolve(r,z,0);
    if (itol == 1)
        err=snrm(r,itol)/bnrm;
    else if (itol == 2)
        err=snrm(z,itol)/bnrm;
    else if (itol == 3 || itol == 4) {
        zminrm=znrm;
        znrm=snrm(z,itol);
        if (abs(zminrm-znrm) > EPS*znrm) {
            dxnrm=abs(ak)*snrm(p,itol);
            err=znrm/abs(zminrm-znrm)*dxnrm;
        } else {
            err=znrm/bnorm;
        }
    }
    xnorm=snrm(x,itol);
    if (err <= 0.5*xnorm) err /= xnorm;
    else {
        err=znrm/bnorm;
    }
}

```

Uncomment this line to get the “minimum residual” variant of the algorithm.

Input to asolve is $r[0..n-1]$, output is $z[0..n-1]$; the final 0 indicates that the matrix $\tilde{\mathbf{A}}$ (not its transpose) is to be used.

Main loop.

Final 1 indicates use of transpose matrix $\tilde{\mathbf{A}}^T$.

Calculate coefficient a_k , new iterate x , and new residuals r and rr .

Solve $\tilde{\mathbf{A}} \cdot \mathbf{z} = \mathbf{r}$ and check stopping criterion.

Error may not be accurate, so loop again.

Error may not be accurate, so loop again.

```

    }
    if (err <= tol) break;
}
}

```

The solve routine uses this short utility for computing vector norms:

```

linbcg.h  Doub Linbcg::snrm(VecDoub_I &sx, const Int itol)
Compute one of two norms for a vector sx[0..n-1], as signaled by itol. Used by solve.
{
    Int i,isamax,n=sx.size();
    Doub ans;
    if (itol <= 3) {
        ans = 0.0;
        for (i=0;i<n;i++) ans += SQR(sx[i]);      Vector magnitude norm.
        return sqrt(ans);
    } else {
        isamax=0;
        for (i=0;i<n;i++) {                      Largest component norm.
            if (abs(sx[i]) > abs(sx[isamax])) isamax=i;
        }
        return abs(sx[isamax]);
    }
}

```

Here is an example of a derived class that solves $\mathbf{A} \cdot \mathbf{x} = \mathbf{b}$ for a matrix $\mathbf{A}$ in NRsparseMat's compressed column sparse format. A naive diagonal preconditioner is used.

```

asolve.h  struct NRsparseLinbcg : Linbcg {
    NRsparseMat &mat;
    Int n;
    NRsparseLinbcg(NRsparseMat &matrix) : mat(matrix), n(mat.nrows) {}
    The constructor just binds a reference to your sparse matrix, making it available to asolve
    and atimes. To solve for a right-hand side, you call this object's solve method, as defined
    in the base class.
    void atimes(VecDoub_I &x, VecDoub_O &r, const Int itrns) {
        if (itrns) r=mat.atx(x);
        else r=mat.ax(x);
    }
    void asolve(VecDoub_I &b, VecDoub_O &x, const Int itrns) {
        Int i,j;
        Doub diag;
        for (i=0;i<n;i++) {
            diag=0.0;
            for (j=mat.col_ptr[i];j<mat.col_ptr[i+1];j++)
                if (mat.row_ind[j] == i) {
                    diag=mat.val[j];
                    break;
                }
            x[i]=(diag != 0.0 ? b[i]/diag : b[i]);
            The matrix  $\tilde{\mathbf{A}}$  is the diagonal part of  $\mathbf{A}$ . Since the transpose matrix has the same
            diagonal, the flag itrns is not used in this example.
        }
    }
};

```

For another example of using a class derived from Linbcg to solve a sparse matrix problem, see §3.8.

CITED REFERENCES AND FURTHER READING:

Tewarson, R.P. 1973, *Sparse Matrices* (New York: Academic Press).[1]